Software Requirements Specification (SRS)

ParkShare — Peer-to-Peer Parking Space Sharing Platform (Working Title)

Version: 0.1 (Draft) **Date:** August 28, 2026 **Prepared for:** Bangladesh market (initial focus: Dhaka) **Status:** Draft — for review and iteration

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for **ParkShare**, a two-sided mobile marketplace connecting **Hosts** (owners of underused private parking spaces) with **Renters** (people needing short- or medium-term parking). It is intended to guide design, development, and testing.

1.2 Scope

In scope (v1):

Android & iOS apps for Hosts and Renters (single app, dual role)
Spot listing, availability scheduling, search & discovery
Instant (on-demand) and recurring (subscription-style) bookings
In-app payments via local gateways
Ratings, verification, and basic dispute handling
Admin/ops web dashboard

Out of scope (v1, candidates for later phases):

Commercial/public parking garage aggregation
EV charging coordination
Underwritten insurance (only partner referrals initially)
Multi-city expansion (launch single-zone first — see §10)

1.3 Definitions

Term	Meaning
Host	User who lists a private parking spot for rent
Renter	User who books a spot
Spot	A single parking space listed by a Host

Term	Meaning
Instant Booking	On-demand booking with little/no advance notice
Commuter Pass	Recurring booking (e.g., weekday 9am–5pm) sold as a subscription
NID	National ID (Bangladesh)
MFS	Mobile Financial Service (bKash, Nagad, Rocket)

1.4 Intended Audience

Founders/product owner, engineering team, QA, early investors, and any legal/compliance advisor brought in later.

2. Overall Description

2.1 Product Perspective

A standalone two-sided marketplace app, structurally similar to Uber/Pathao's matching model, but the "asset" being shared is a **static real-estate resource** (a parking spot) rather than a moving vehicle. This changes several things: no live driver-tracking core loop, but *access control* (getting the renter physically into a private space) becomes a first-class problem.

2.2 User Classes

Host — lists spot(s), sets availability & price, approves/monitors bookings
Renter — searches, books, pays, checks in/out
Admin/Ops — moderation, verification, dispute resolution, analytics
(Later) Field Verification Agent — physically confirms new listings

2.3 Operating Environment

Cross-platform mobile (React Native or Flutter recommended for a lean team), cloud backend (AWS/GCP, region latency-optimized for South Asia), designed to work acceptably on 3G/4G and budget Android devices, which dominate the Bangladesh market.

2.4 Design & Implementation Constraints

Must integrate **bKash, Nagad, Rocket**, plus card payments (via SSLCommerz or ShurjoPay) — cash-only UX will not scale trust or reconciliation.
Bangla + English localization required.
OTP-based phone authentication (email-first onboarding will underperform here).
Low-bandwidth-friendly design (compressed images, lightweight map tiles).

2.5 Assumptions & Dependencies

Hosts have legitimate rights to sublet their allotted parking (see §9 — this is **not** guaranteed and needs legal review).
Maps provider (Google Maps Platform or Mapbox) is available and its per-request cost is sustainable at scale.
Renters own/have access to a smartphone with GPS.

3. Functional Requirements

3.1 User Management

FR1.1 Register/login via phone number + OTP
FR1.2 Upload NID or Driving License for identity verification (both roles)
FR1.3 Add vehicle(s): plate number, type, size class
FR1.4 Single account can act as Host, Renter, or both, with a role switch in-app

3.2 Host — Spot Listing

FR2.1 Add spot with GPS pin + address
FR2.2 Spot metadata: covered/open, dimensions/vehicle-size supported, access type (gate code / guard / remote), photos
FR2.3 Set **recurring availability** (e.g., Mon–Fri, 9:00–19:00)
FR2.4 Set **one-off availability** for ad-hoc listing
FR2.5 Set pricing (hourly / daily / monthly), with platform-suggested price ranges by area
FR2.6 Edit/block availability in real time (e.g., host returns home early)
FR2.7 Approve bookings manually or enable auto-approve

3.3 Renter — Search & Booking

FR3.1 Map + list search near a destination (current location, address, or "near X restaurant")
FR3.2 Filter by price, distance, covered/open, time window
FR3.3 View spot detail, host rating, and photos before booking
FR3.4 Instant Booking flow (short-notice, e.g., restaurant use case)
FR3.5 Commuter Pass flow (recurring/subscription booking)
FR3.6 In-app turn-by-turn navigation to the spot
FR3.7 Cancel/reschedule, governed by cancellation policy

3.4 Access & Verification

FR4.1 QR code or PIN-based check-in/check-out at the spot
FR4.2 Optional geofencing to auto-detect arrival/departure (for usage-based billing)
FR4.3 Push notification to Host on Renter check-in/check-out
FR4.4 In-app SOS/emergency contact button

3.5 Payments & Wallet

FR5.1 Integrate bKash, Nagad, Rocket, and cards (via SSLCommerz/ShurjoPay)
FR5.2 In-app wallet: prepaid balance for Renters, earnings + withdrawal-to-MFS/bank for Hosts
FR5.3 Automatic fare calculation (booked duration, or actual duration via check-in/out)
FR5.4 Optional security deposit/hold for high-value or long bookings
FR5.5 Digital invoice/receipt per booking
FR5.6 Platform commission logic (deducted from Host payout)

3.6 Trust, Ratings & Safety

FR6.1 Two-way rating/review after each booking
FR6.2 "Verified" badge for Hosts with confirmed ID + physically confirmed spot
FR6.3 Report/flag a user, listing, or booking

3.7 Notifications & Communication

FR7.1 Push notifications: booking confirmed, reminders, cancellations, disputes
FR7.2 In-app masked chat/call between Host and Renter (numbers not exposed directly)

3.8 Admin & Operations

FR8.1 Admin dashboard: user & listing management, moderation
FR8.2 Analytics: bookings, revenue, active spots, demand heatmap by area
FR8.3 Manual verification workflow for new Hosts (especially pre-scale)
FR8.4 Dispute intake and refund tools

3.9 Special Booking Modes

FR9.1 Commuter Pass — recurring weekday booking near a workplace, subscription-priced
FR9.2 Instant Spot — on-demand short-term booking (restaurants, events), minimal lead time
FR9.3 *(Phase 2)* Dynamic pricing engine based on time/location demand

4. Non-Functional Requirements

Category	Requirement
Performance	Search results return in <2s; booking confirmation <5s
Scalability	Must handle city-wide (Dhaka) load; horizontally scalable backend
Availability	99.5% uptime target

Category	Requirement
Security	Encrypted storage of NID/payment data; no raw card data stored (delegate to gateway); secure OTP auth
Usability	Bangla + English UI; simple, low-literacy-friendly design
Reliability	Booking transactions must be atomic — **no double-booking** of the same spot/time slot
Compatibility	Android 8+, iOS 13+; optimized for low/mid-range devices
Legal/Regulatory	Data handling aligned with Bangladesh's Digital Security/ICT regulatory framework; ToS should clarify platform is a facilitator, not guarantor, of listing legality (see §9)

5. External Interface Requirements

UI: Single app (Host/Renter mode toggle) + web Admin dashboard
Hardware: GPS, camera (QR scanning, photo verification)
Software/APIs: Maps (Google Maps Platform / Mapbox), Payment gateways (bKash, Nagad, SSLCommerz/ShurjoPay), local SMS/OTP aggregator, Firebase Cloud Messaging (push)
Communication: REST/GraphQL over HTTPS/TLS

6. High-Level System Architecture

Client: React Native or Flutter (single codebase, cost-effective for a small team)
Backend: Modular monolith or microservices (Node.js/NestJS or similar), REST/GraphQL API
Database: PostgreSQL (transactional integrity for bookings/payments) + PostGIS (geospatial "spots near me" queries) + Redis (caching, real-time availability)
Integrations: Maps API, payment gateways, SMS/OTP, push notifications
Admin: Web dashboard (React)

7. Core Data Entities

User — id, name, phone, NID doc, role, rating, wallet balance
Vehicle — id, user_id, plate, type, size class
ParkingSpot — id, host_id, lat/lng, address, access_type, dimensions, photos, verified_status
Availability — spot_id, recurring_rule or date range, price
Booking — id, spot_id, renter_id, start/end time, status, type (instant/recurring), amount
Transaction — id, booking_id, amount, method, status

Review — id, booking_id, from_user, to_user, rating, comment
Dispute — id, booking_id, raised_by, status, resolution

8. Key Use Case Scenarios

Host lists a spot — Owner adds home parking, sets weekday daytime availability, sets price, uploads photos.
Commuter books recurring spot — Office worker finds a spot near work, subscribes to a weekday Commuter Pass, checks in/out daily via QR.
Instant restaurant booking — Renter searches near a restaurant an hour before dinner, books instantly, walks over after parking.
Host cancels last-minute — Host returns home early and needs to cancel/shorten an active listing; system needs a fair notice/penalty policy for the affected Renter.

9. Assumptions, Risks & Open Questions

Legal risk (important): In many Dhaka apartment/office buildings, allotted parking may be governed by building bylaws, RAJUK regulations, or society rules that restrict subletting. This should be reviewed with a lawyer before launch — it's a foundational risk, not a minor detail.
Trust & physical safety risk: Renters gain access to strangers' private property (driveways, building compounds). ID verification alone won't fully cover this — see suggestions below.
Marketplace cold-start problem: Two-sided marketplaces are hard to bootstrap — need Hosts and Renters in the *same small area* simultaneously.
Access control: Many homes/buildings have gated compounds with security guards — the app needs a real answer for "how does the Renter's car physically get in," not just a booking confirmation.
Payment gateway settlement time/fees in Bangladesh should be budgeted into the commission model early.

10. Suggested MVP Phasing

Phase 0 — Manual Pilot: Match Hosts and Renters manually (WhatsApp/Google Form) in one neighborhood to validate real demand before writing code.
Phase 1 — Commuter Pass only: Recurring bookings only, single dense zone (e.g., a commercial pocket like Gulshan/Banani/Motijheel). Lower fraud risk, predictable supply/demand, easier to reason about liability.
Phase 2 — Instant/On-demand: Add the restaurant/event use case once trust mechanisms and check-in/out flows are proven.
Phase 3 — Scale: Dynamic pricing, insurance partnerships, B2B (companies renting bulk spots for staff), multi-city expansion.